

Exercise 2 – Evaluation of Skin Color Detection

1. Operation of the Algorithm

The developed algorithm functions by converting the video stream from RGB to HSV color space and applying a specific threshold to filter skin-tone pixels. Post-processing steps (Downscaling/Upscaling and Median Filtering) are applied to reduce noise and fill gaps.

Below are examples of the algorithm in operation:

Example 1: Hand Detection

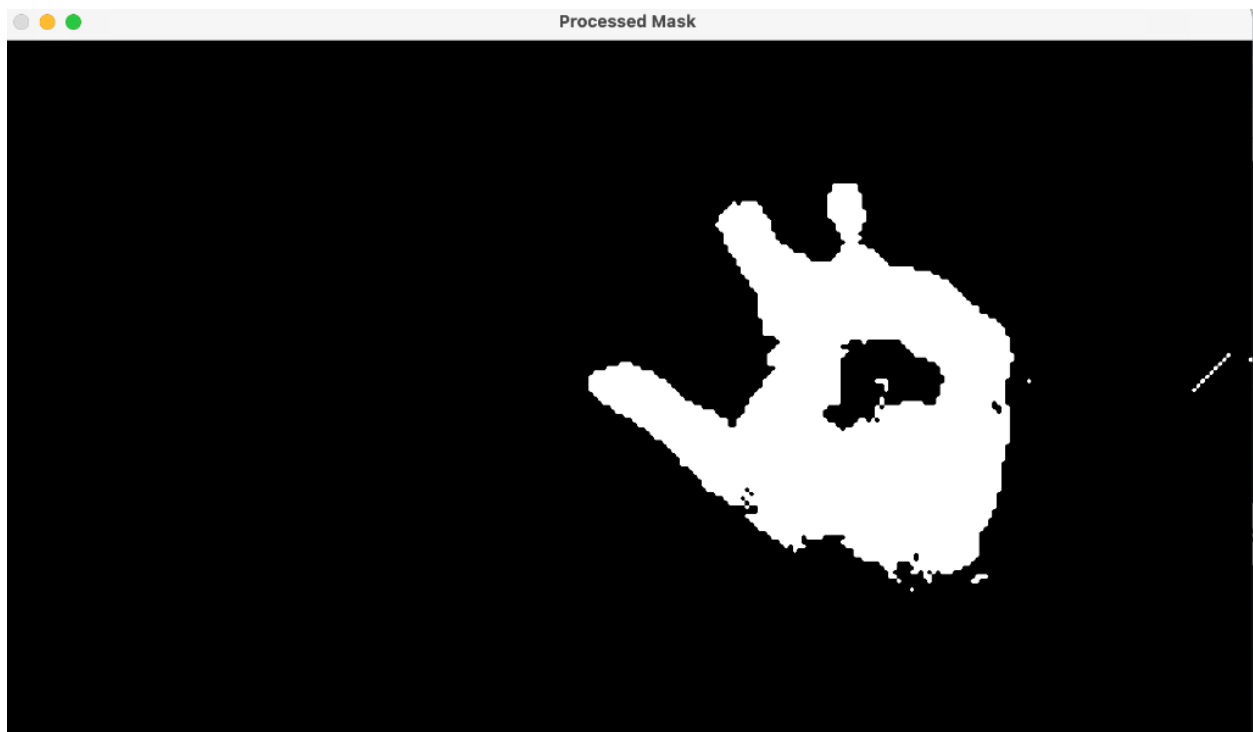


Figure 1: Successful detection of a hand gesture. The mask clearly captures the shape of the hand.

Example 2: Face and Hand Detection



Figure 2: Simultaneous detection of face and hand. Note that both skin regions are highlighted in white.

Example 3: Noise / Background Interference



Figure 3: Example showing potential background noise where small artifacts might appear if the background contains skin-like colors.

2. Pros and Cons of the Method

Pros

- **Computationally Efficient:** The algorithm relies on simple pixel-wise thresholding, making it extremely fast and suitable for real-time applications with low latency.
- **Rotation Invariant:** Since the detection is based on color statistics (pixel values), the orientation of the hand or face does not affect detection (unlike template matching or Haar cascades).
- **Implementation Simplicity:** Does not require training data or complex machine learning models; it is purely mathematical and rule-based.

Cons

- **Lighting Sensitivity:** The HSV model is still susceptible to lighting changes. Deep shadows or very bright highlights can shift pixel values outside the defined threshold range ($0.2 \leq S \leq 0.6$, etc.), causing partial occlusion of the mask.
- **Background Confusion:** Any object in the background with a color similar to skin (e.g., wood, beige walls, cardboard) will be detected as a false positive.
- **No Shape Knowledge:** The algorithm only knows "color," not "structure." It cannot distinguish between a hand, a face, or a beige cup.

3. Effectiveness and Sufficiency

Is the skin color method sufficient? For a controlled environment with uniform lighting and a contrasting background, the method is **sufficient** for basic interactions. It successfully isolates the user's input mechanism (hand/head).

Is it effective? Its effectiveness is **limited** in real-world scenarios. In a typical room with variable lighting and cluttered backgrounds, the method suffers from noise and instability. Without additional constraints (like motion tracking or shape analysis), it is not robust enough for precise cursor control on its own.

4. Solving the "Head vs. Hand" Problem

The Problem: When both the head and hand are visible (as seen in Figure 2), the resulting binary mask contains two large white blobs. If we naively calculate the centroid

(center of mass) of all white pixels, the "mouse pointer" would end up floating somewhere in the empty space between the face and the hand.

Proposed Solutions:

1. Largest Connected Component (Blob Analysis):

- **Method:** Use an algorithm (like `cv2.findContours`) to identify separate "islands" of white pixels.
- **Logic:** Calculate the area of each blob. Assume that the **Face** is the largest object (if the user is close) or that the **Hand** is the largest object (if the hand is held closer to the camera).
- **Steering:** If we want to steer with the hand, we can instruct the user to hold their hand closer to the camera so it becomes the largest specific blob.

2. Positional Heuristics:

- **Method:** Distinguish blobs based on their Y-coordinate.
- **Logic:** The **Face** is typically in the upper half of the frame, while the **Hand** moves in the lower half or to the side. We can set a region of interest (ROI) or simply select the "lowest" large blob to represent the hand.

3. Motion Tracking (Optical Flow):

- **Method:** Assume the head is relatively stationary while the hand moves to control the mouse.
- **Logic:** Calculate the difference between consecutive frames. The blob with the most movement (pixel changes) is identified as the controller.

4. Shape/Aspect Ratio Analysis:

- **Method:** Analyze the bounding box of each blob.
- **Logic:** A face is generally oval (aspect ratio $\sim 1:1.3$). An open hand might have a different aspect ratio or complexity (number of convexity defects between fingers). This is more complex but more accurate.

Conclusion: The most robust simple solution is **Connected Component Analysis**. By identifying individual blobs and selecting the one that matches our criteria (e.g., "The blob that is currently moving" or "The largest blob"), we can ignore the other skin-colored regions.